

Design card — group 21 (seher-suhani) — switch_21

Approach: link-state routing over the switch API. HELLO discovers neighbors and doubles as a heartbeat. Flooded state advertisements (ADV) give every switch the full topology and prefix ownership. Each switch runs its own BFS to pick a next-hop port. The data plane is two dumb tables; the control plane is the only thing that ever writes to them.

Protocol

	HELLO (proto 89)	ADV (proto 90)
sent	every 50ms, every port	on any local change + every ~1s refresh
ttl	1 (one hop only)	1 (re-injected fresh at each hop)
payload	switch_id (4B)	origin(4B) seq(4B) neighbor_count(2B) neighbors(4B each) has_prefix(1B) prefix_network(4B)
does	tells us who's on the other end of each cable; silence for 3 ticks (~150ms) = link is dead	carries "my current live neighbors + my prefix, if known"; receiver keeps only the newest seq per origin and re-floods it out every other port

Prefix learning: nobody tells a switch what's behind it. The first data packet with no route (NoRoute) arriving on a port that has never produced a HELLO is assumed to be our own customer app — we read its source /24 and fold it into our next ADV.

Routing: BFS from self over the graph built from every ADV collected (including our own). For each destination switch, "which of my neighbors starts the shortest path there" gives the egress port for that switch's prefix.

Data plane

Table	Kind	Key	Purpose
T_PROTO (1)	Exact	ip_proto	catches 89/90 before routing sees them, punts Custom(proto)
T_ROUTE (2)	LPM /24	ip_dst	real forwarding table; entry id = prefix network address; miss punts NoRoute

Route changes are always delete-then-install, never a bare install over an existing id — install() only appends, so a stale entry could still win.

Key parameters

Parameter	Value	Why
HELLO tick	50ms	fast detection, negligible next to 1Gbps links
dead-neighbor threshold	3 missed ticks (~150ms)	avoids declaring dead on one lost packet; well inside the 1000ms budget
ADV periodic refresh	every ~1s	backstop if a flooded ADV is itself lost
measured recovery (15/15 Part 2 practice runs)	~150-200ms worst case	detection dominates; convergence itself is sub-millisecond
scoring window	warmup 200ms, ends 100ms before run stops	losses in that window don't count against us

Known limitation: transient microloop during convergence

Seen in part2-ring-010-f001: link 2-14 failed; switches 2 and 10 each independently recomputed their route to switch 14's prefix and installed the new entries 123 microseconds apart. In that window switch 2 routed *through* 10 while switch 10 still routed *through* 2 — one in-flight packet bounced once (s10->s2->s10) before escaping on switch 10's very next, now-correct, lookup.

- Why: each switch installs its own new decision the instant its own flood recompute fires, with no coordination on timing between switches. Once everyone has heard the same news the result is globally consistent and loop-free by construction — the loop only lives in the seam while the flood is still spreading.
- Why it didn't run away: convergence is fast enough that it self-corrected in microseconds; TTL decrement is the real backstop if it hadn't.
- How I'd patch it (not done — ungraded until A2): make-before-break, or hold the old route briefly past when the new one is computed.

Reading our own evidence

- Which path is traffic on? Look up the destination /24 in T_ROUTE's egress port, cross-reference against the neighbor table to name the next switch, repeat there.
- Why did switch X do Y at time T? Its decision is a pure function of node_state at that instant — check the newest ADV/HELLO it had heard before T.
- What if link A-B had failed instead of C-D? Remove that edge from the topology by hand and re-run BFS from the affected switch; only switches whose shortest path used that edge reroute.
- How would you break this network? No authentication on ADV — a forged high-seq advertisement for someone else's switch id would be believed and could blackhole that prefix.

Fill in after the graded run: switch_id to owner-port mapping for that world, and anything you personally keep forgetting under quiz pressure.